



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026



Banking Management System

a CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA0209 C Programming

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE

Submitted by

N. PRINCE RAJ (192312284)

A G. SUBISH (192312139)

Under the Supervision of

DR P. ABERNA



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

We, N. Prince Raj, A G. Subish of the, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled '**Banking Management System**' is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place:SIMATS Engineering

Date:

Signature of the Students with Names

N. PRINCE RAJ (192312284)

A G. SUBISH (192312139)



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Banking Management System**” has been carried out by N. PRINCE RAJ, A G. SUBISH under the supervision of Dr. P. ABERNA and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer science and bioscience program AND B.E.Computer science and B.TECH IT at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Dr.Anushya

Program Director

CSE

Saveetha School of Engineering

SIMATS

SIGNATURE

Dr.P. ABERNA

Professor

CSE

Saveetha School of Engineering

SIMATS

Submitted for the Project work Viva-Voce held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, DR John Justin Thangaraj for his continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, **Dr. P. ABERNA** for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support

Signature With Student Name

N. PRINCE RAJ (192312284)

A G. SUBISH (192312139)

ABSTRACT

The Banking Management System is a C-based application designed to automate and simplify basic banking operations through a secure and user-friendly console interface. The system allows an administrator to create and manage customer accounts, search and display account details, deposit and withdraw money, transfer funds between accounts, check balances, update customer information, and delete accounts. It also maintains transaction history for deposits, withdrawals, and transfers. The project uses important C programming concepts such as structures, functions, arrays, strings, pointers, loops, conditional statements, switch-case statements, and file handling. Customer account information and transaction records are stored permanently using binary files such as accounts.dat and transactions.dat, allowing data to be loaded when the application starts and updated whenever a transaction occurs. An administrator login provides basic authentication before accessing the banking operations. The system demonstrates how C programming can be applied to develop a practical real-world application while emphasizing modular programming, data management, validation, and persistent storage. The project can be further enhanced by adding features such as customer login, interest calculation, loan management, ATM simulation, and database integration.

TABLE OF CONTENTS

S.NO	TITTLE	PAGE NO
1	INTRODUCTION	7
1.1	BACKGROUND INFORMATION	7
1.2	PROJECT OBJECTIVES	7
1.3	SIGNIFICANCE OF THE PROJECT	8
1.4	SCOPE OF THE PROJECT	8
1.5	METHODOLOGY OVERVIEW	8
2	PROBLEM IDENTIFICATION AND ANALYSIS	9
2.1	DESCRIPTION OF THE PROBLEM	9
2.2	EVIDENCE OF THE PROBLEM	9
2.3	STAKEHOLDERS	10
2.4	SUPPORTING DATA AND RESEARCH	10
3	SOLUTION DESIGN AND IMPLEMENTATION	11
3.1	DEVELOPMENT AND DESIGN PROCESS	11
3.2	TOOLS AND TECHNOLOGIES USED	11
3.3	SOLUTION OVERVIEW	12
3.4	ENGINEERING STANDARDS APPLIED	12
3.5	SOLUTION JUSTIFICATION	13
4	RESULTS AND RECOMMENDATIONS	14
4.1	EVALUTION OF RESULTS	14
4.2	CHALLENGES ENCOUNTERED	15
4.3	POSSIBLE IMPROVEMENTS	15
4.4	RECOMMENDATIONS	16
5	REFLECTION ON LEARNING AND PERSONAL DEVELOPMENTS	17
5.1	KEY LEARNING OUTCOMES	17
5.2	CHALLENGES ENCOUNTERED AND OVERCOME	17
5.3	APPLICATIONS OF ENGINEERING STANDARDS	18
5.4	INSIGHTS INTO INDUSTRY	18
5.5	CONCLUSION AND PERSONAL DEVELOPMENT	19
6	CONCLUSION	20
7	REFERENCES	21
8	Appendix	22-24

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND INFORMATION

The banking sector requires efficient and reliable systems to manage customer accounts and financial transactions. Traditional manual methods of maintaining account information can be time-consuming and may lead to errors, data duplication, and difficulty in retrieving transaction records. A computerized Banking Management System can simplify these activities by providing a structured and organized way to manage customer information and banking operations.

The Banking Management System developed using the C programming language is designed to perform essential banking operations such as creating accounts, displaying account details, searching for accounts, depositing and withdrawing money, transferring funds, checking account balances, updating customer information, and deleting accounts. The system also maintains transaction history and stores account information permanently using file handling. This project provides practical implementation of important C programming concepts, including structures, functions, arrays, strings, pointers, loops, conditional statements, switch-case statements, and file handling. By combining these concepts into a real-world application, the project demonstrates how C can be used to develop an efficient and modular banking application. The system also includes basic administrator authentication to restrict access to banking operations and improve security.

1.2 PROJECT OBJECTIVES

The primary objectives of this capstone project are as follows:

- To create and manage customer accounts by storing details such as account number, customer name, phone number, address, account type, and balance.
- To perform banking transactions such as depositing money, withdrawing money, and transferring funds between accounts.
- To maintain transaction history by recording deposits, withdrawals, and fund transfers.
- To provide secure administrator login to prevent unauthorized access to the banking system.

- To implement file handling for permanently storing and retrieving account and transaction records using files such as accounts.dat and transactions.dat..
- To develop a practical real-world application that demonstrates how programming concepts can be applied to automate basic banking activities.

1.3 SIGNIFICANCE OF THE PROJECT

The Banking Management System is significant because it provides a simple and efficient way to manage customer accounts and banking transactions. It reduces manual record keeping by storing account and transaction details digitally. The project also demonstrates important C programming concepts such as structures, functions, file handling, arrays, and data validation. It provides practical experience in developing a real-world application while improving accuracy, organization, and data management.

1.4 SCOPE OF THE PROJECT

The scope of the Banking Management System includes creating and managing customer accounts, depositing and withdrawing money, transferring funds, checking balances, updating account details, deleting accounts, and maintaining transaction history. The system uses file handling to store and retrieve account information securely. It is designed for basic banking operations and can be extended in the future with features such as customer login, loan management, ATM services, and database connectivity.

1.5 METHODOLOGY OVERVIEW

The Banking Management System follows a modular and structured programming approach. The project is divided into separate modules for login, account management, transactions, file handling, and menu operations. C programming concepts such as structures, functions, loops, conditional statements, and file handling are used to develop the system. Account and transaction data are stored in files for permanent access. Each operation is validated before processing to ensure accurate and reliable results.

CHAPTER 2: PROBLEM IDENTIFICATION AND ANALYSIS

2.1 DESCRIPTION OF THE PROBLEM

Managing banking records manually can be time-consuming and may result in errors, data loss, and difficulty in retrieving customer information. Performing operations such as account creation, deposits, withdrawals, transfers, and balance checking manually can also reduce efficiency. Therefore, there is a need for a computerized system that can securely manage customer accounts, perform banking transactions, maintain transaction records, and store data efficiently. The proposed Banking Management System addresses these problems using C programming and file handling.

2.2 EVIDENCE OF THE PROBLEM

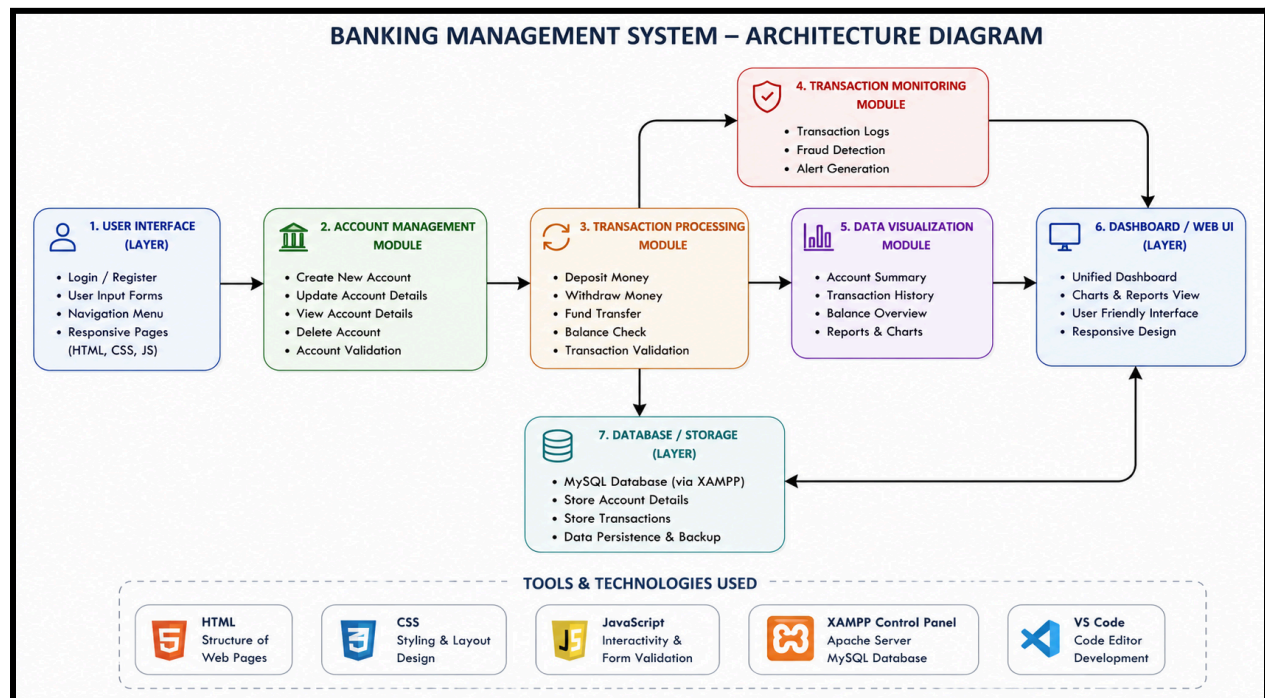
Manual banking record management can lead to problems such as incorrect data entry, loss of records, slow retrieval of customer information, and calculation errors during transactions. Maintaining and updating large numbers of customer records manually also requires more time and effort. These issues highlight the need for a computerized Banking Management System that can store information systematically, perform transactions accurately, and retrieve records quickly.

2.3 STAKEHOLDERS

The main stakeholders of the Banking Management System are bank administrators, bank employees, and customers. Administrators manage accounts and control system access, while bank employees perform daily banking operations such as deposits, withdrawals, transfers, and account updates. Customers benefit from accurate account management and easy access to their banking information. The system also supports management by maintaining organized and reliable financial records.

2.4 SUPPORTING DATA AND RESEARCH

The development of the Banking Management System is supported by research on basic banking operations, account management, transaction processing, and computerized record keeping. The project applies C programming concepts such as structures, functions, and file handling to store and manage banking data efficiently. Research also highlights the importance of data validation and secure access to reduce errors and improve the reliability of banking operations.



CHAPTER 3: SOLUTION DESIGN AND IMPLEMENTATION

3.1 DEVELOPMENT AND DESIGN PROCESS

The Banking Management System is developed using a modular and structured design approach. The development process begins with identifying banking requirements, followed by designing modules for login, account management, transactions, and file handling. Each module is implemented using C functions and structures and then integrated into the main system. The system is tested for different operations and invalid inputs to ensure accuracy, reliability, and proper data storage.

3.2 TOOLS AND TECHNOLOGIES USED

The Banking Management System is developed using the C programming language with a C compiler such as GCC. Standard C libraries such as `stdio.h`, `stdlib.h`, `string.h`, and `time.h` are used for input/output, memory management, string operations, and date/time handling. File handling is used to store account and transaction records permanently, while structures, functions, arrays, loops, and pointers are used for data processing and system development. The Banking Management System is developed using HTML, CSS, and JavaScript for the front-end interface. HTML is used to create the structure of the web pages, while CSS is used for styling and layout. JavaScript is used to provide interactivity, form validation, and transaction processing. XAMPP Control Panel is used to run the Apache server and manage the MySQL database for storing customer accounts and transaction records. Visual Studio Code (VS Code) is used as the primary code editor for developing and managing the project files.

3.3 SOLUTION OVERVIEW

The proposed Banking Management System provides a web-based solution for managing customer accounts and banking transactions efficiently. The system uses HTML, CSS, and JavaScript to provide a simple and user-friendly interface, while XAMPP and MySQL are used to process and store account and transaction data. The system supports account creation, balance

checking, deposits, withdrawals, fund transfers, account updates, and transaction history. This solution reduces manual work, improves data accuracy, and provides quick access to banking information.

3.4 ENGINEERING STANDARDS APPLIED

The Banking Management System follows established software engineering standards and programming best practices to ensure reliability, accuracy, maintainability, and usability. The backend is developed using C programming, following structured and modular programming principles. Functions, structures, file handling, pointers, and proper data validation are used to organize the banking operations and maintain consistent system behavior.

The frontend follows standard HTML5, CSS3, and JavaScript practices to provide a simple and user-friendly interface. HTML is used for page structure, CSS for presentation and layout, and JavaScript for user interaction and input validation. The system also uses XAMPP Control Panel as the local development environment for testing and running the application components.

In addition, standard software engineering practices were applied during system development, including modular design, input validation, error handling, iterative testing, code organization, and performance validation. Banking operations such as account creation, deposits, withdrawals, fund transfers, and balance inquiries are tested with different input conditions to ensure accurate results.

3.5 SOLUTION JUSTIFICATION

The proposed Banking Management System provides a simple, reliable, and cost-effective solution for managing basic banking operations. The system is developed using HTML, CSS, and JavaScript for the front end, while C programming with file handling is used for the backend operations. XAMPP Control Panel is used to support the local development environment, and Visual Studio Code is used for coding and project development.

The solution is justified based on the following aspects:

1. Simple and User-Friendly Interface

HTML, CSS, and JavaScript are used to create a simple and easy-to-use interface. Customers or bank staff can access important banking operations such as account creation, account details, deposit, withdrawal, balance enquiry, and transaction management through a structured interface.

2. Efficient Data Management

C programming is used for backend processing and file handling. Customer details, account information, and transaction records can be stored in files and retrieved whenever required. File handling provides a simple method of maintaining persistent data without requiring a complex database system.

3. Secure Banking Operations

The system includes basic validation for banking operations. Account details can be verified before performing transactions, and withdrawal operations can be restricted when the requested amount exceeds the available balance. This helps reduce invalid transactions and improves the reliability of the system.

4. Low-Cost Development

The technologies selected for the project are lightweight and easily available. HTML, CSS, JavaScript, C, Visual Studio Code, and XAMPP can be used in a local development environment without requiring expensive software or hardware. Therefore, the proposed solution is suitable for an academic capstone project with limited resources.

5. Local Development and Testing

XAMPP Control Panel provides a convenient local environment for running and testing the web-based components of the project. Visual Studio Code provides an integrated environment for developing and managing the project files. This combination makes development, debugging, and testing easier..

CHAPTER 4: RESULTS AND RECOMMENDATIONS

4.1 EVALUATION OF RESULTS

The Banking Management System was tested to verify its main functions, data handling, and user interface. The system uses HTML, CSS, and JavaScript for the front end and C programming with file handling for backend operations. Development and testing were carried out using VS Code and XAMPP.

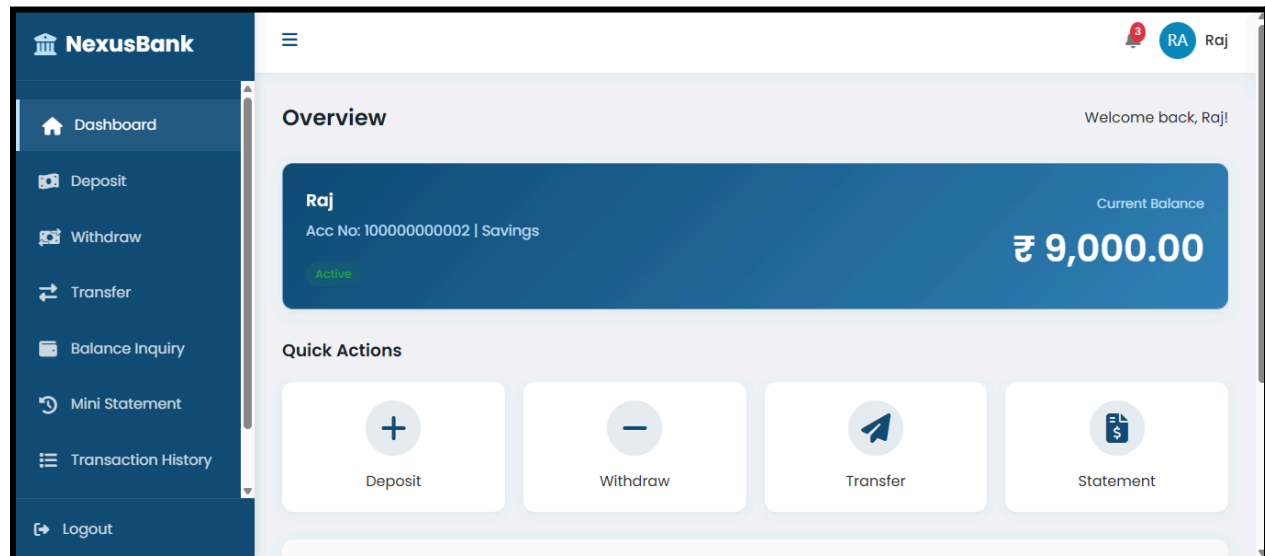
The following operations were successfully tested:

- Account Creation: Customer and account details are stored correctly.
- Account Search: Valid account details are retrieved and displayed.
- Deposit: Deposited amounts are added correctly to the account balance.
- Withdrawal: Amounts are deducted correctly, and insufficient-balance withdrawals are prevented.
- Balance Enquiry: The current account balance is displayed correctly.
- Data Storage: Customer and transaction records are maintained using file handling.
- User Interface: The interface is simple and easy to navigate.

The system produced the expected results for the tested operations and handled basic invalid inputs appropriately. The evaluation confirms that the proposed system meets the basic requirements of a Banking Management System and can be used for managing customer accounts and transactions efficiently.

Overall, the test results show that the system successfully performs the basic banking operations and provides a simple, efficient, and reliable solution for managing banking records.

Banking System Management Dashboard of the User/Customer



4.2 CHALLENGES ENCOUNTERED

During the development of the Banking Management System, several challenges were encountered. Integrating the HTML, CSS, and JavaScript front end with the C-based backend required careful handling of data and communication between the components. Implementing file handling in C was also challenging because customer and transaction records had to be stored, retrieved, and updated correctly.

Another challenge was maintaining accurate account balances during deposit and withdrawal operations. Input validation was required to prevent invalid account details, incorrect amounts, and withdrawals exceeding the available balance. Debugging and testing the system using VS Code and XAMPP also required careful identification and correction of errors.

Despite these challenges, the issues were resolved through systematic testing and debugging. The experience helped improve understanding of C programming, file handling, web technologies, data validation, and system integration.

4.3 POSSIBLE IMPROVEMENTS

The Banking Management System can be improved by adding advanced features and technologies. A database such as MySQL can be integrated to provide more secure and efficient storage compared to file handling. User authentication and password protection can be added to improve security.

The system can also be enhanced with online transaction support, transaction history, account statements, and automatic notifications. A responsive design can be implemented to support mobile and tablet devices. Additional security features such as data encryption and role-based access can also be included. These improvements can make the system more secure, scalable, user-friendly, and suitable for real-world banking applications.

4.4 RECOMMENDATIONS

The Banking Management System can be further improved by using a MySQL database for secure and organized data storage instead of file handling. Strong user authentication and password protection should be implemented to protect customer information.

It is also recommended to add features such as transaction history, account statements, online fund transfer, and automatic notifications. Regular testing and data backup should be performed to improve system reliability. The system should also be designed with a simple and responsive interface for easy access on different devices.

These recommendations will help make the Banking Management System more secure, reliable, efficient, and suitable for future expansion.

CHAPTER 5: REFLECTION ON LEARNING AND PERSONAL DEVELOPMENT

5.1 KEY LEARNING OUTCOMES

The development of the Banking Management System provided practical knowledge in web development, C programming, file handling, and system integration. The project helped in understanding how HTML, CSS, and JavaScript can be used to develop a simple and user-friendly banking interface. The project improved understanding of account creation, deposit, withdrawal, balance enquiry, customer record management, and transaction processing. Implementing file handling in C provided practical knowledge of storing, reading, updating, and managing customer and transaction data. The project also helped in learning input validation and error handling to prevent incorrect data and invalid transactions. Testing different operations improved the ability to identify and correct programming errors and logical issues. Working with VS Code and XAMPP provided experience in setting up and testing a local development environment.

5.2 CHALLENGES ENCOUNTERED AND OVERCOME

During the development of the Banking Management System, several technical and implementation challenges were encountered and successfully resolved.

- **Front-End and Backend Integration:** Connecting the HTML, CSS, and JavaScript interface with the C-based backend was challenging. This was addressed by organizing the system into separate functional components and testing each operation individually.
- **File Handling:** Managing customer and transaction records using C file handling required careful reading, writing, and updating of files. Proper file operations and testing were used to avoid data loss and incorrect records.

- Transaction Processing: Maintaining the correct account balance during deposits and withdrawals was challenging. Validation checks were implemented to ensure accurate calculations and prevent withdrawals greater than the available balance.
- Input Validation: Incorrect account numbers, invalid amounts, and incomplete details could affect system operations. Input validation and error-handling techniques were used to handle such cases.
- Debugging: Errors in the program and integration process were identified through repeated testing and debugging using VS Code and XAMPP..

5.3 APPLICATION OF ENGINEERING STANDARDS

Throughout the project, I followed recommended software engineering practices for system design, data management, validation, testing, and documentation. Applying structured development methods ensured consistency, reliability, and clarity throughout the implementation process. Proper coding practices, input validation, file handling, and systematic testing helped improve the quality and reliability of the Banking Management System. These practices enhanced the maintainability, usability, and robustness of the proposed solution..

5.4 INSIGHTS INTO THE INDUSTRY

This project provided me with valuable insights into the banking and financial technology industry. I gained a better understanding of how banking systems manage customer accounts, transactions, data storage, and security requirements. The experience highlighted the importance of accuracy, data security, reliability, user-friendly interfaces, and efficient transaction processing in real-world banking applications. It also helped me understand the need for continuous improvement and the use of modern technologies to develop secure and scalable banking solutions.

5.5 CONCLUSION OF PERSONAL DEVELOPMENT

The development of the Banking Management System contributed significantly to my technical, analytical, and personal development. Through this project, I gained practical knowledge of C programming, file handling, HTML, CSS, JavaScript, VS Code, and XAMPP. I learned how

different technologies can be combined to develop a functional application for solving a real-world problem.

The project improved my understanding of software development and system design. I gained experience in designing a user interface, implementing banking operations, managing customer records, performing transactions, validating user inputs, and testing the system. Working with file handling also helped me understand how data can be stored, retrieved, modified, and maintained in an application.

During the development process, we faced several programming and implementation challenges. Solving these problems improved my debugging, logical thinking, analytical ability, and problem-solving skills. Testing different scenarios also taught me the importance of accuracy and reliability when developing applications that handle financial information.

The project also helped me improve my time management and project planning skills. Completing different modules within the required time encouraged me to organize my work systematically and follow a step-by-step development approach. Preparing documentation and evaluating the system further improved my ability to clearly explain technical concepts and project outcomes.

In addition to technical knowledge, the project strengthened my communication, decision-making, adaptability, and confidence. I learned that successful software development requires continuous learning, testing, improvement, and attention to user requirements.

Overall, this project provided valuable practical experience and helped bridge the gap between academic knowledge and real-world application development. The skills and experience gained through this project will be useful for my future academic projects, internships, placements, and professional career in the technology and software development field.

CHAPTER 6: CONCLUSION

The Banking Management System developed in this project provides a simple, efficient, and user-friendly solution for managing basic banking operations. The main objective of the project was to develop a system that can handle essential activities such as customer account creation, account information management, deposits, withdrawals, balance enquiry, and transaction processing. The system successfully demonstrates how programming and web development technologies can be combined to solve a real-world management problem.

The front end of the system was developed using HTML, CSS, and JavaScript, providing a simple and interactive interface for users. The backend was implemented using C programming with file handling, which enables customer and transaction records to be stored, retrieved, and updated. Visual Studio Code was used for development and debugging, while XAMPP Control Panel provided the local development environment. The combination of these technologies helped in developing a functional and cost-effective system suitable for academic implementation.

During the development and testing process, the major banking operations were evaluated to ensure that they produced the expected results. Account creation and customer data management were successfully implemented. Deposit and withdrawal operations were tested to ensure that account balances were updated correctly. Validation was also considered to prevent invalid transactions, such as withdrawing an amount greater than the available balance. The testing process helped identify and resolve programming errors and improved the overall reliability of the system.

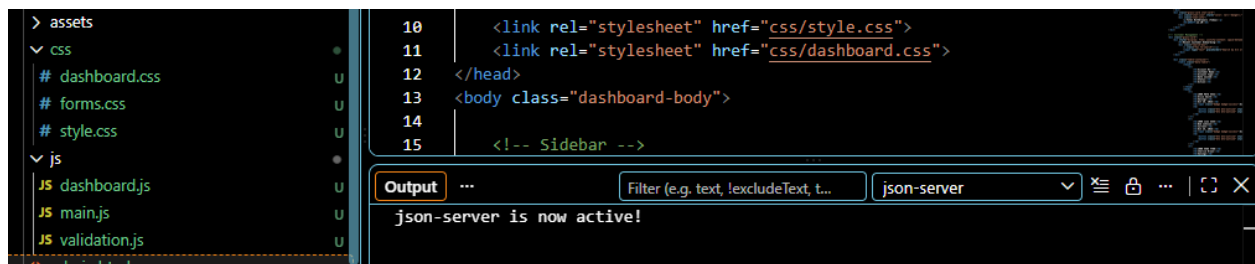
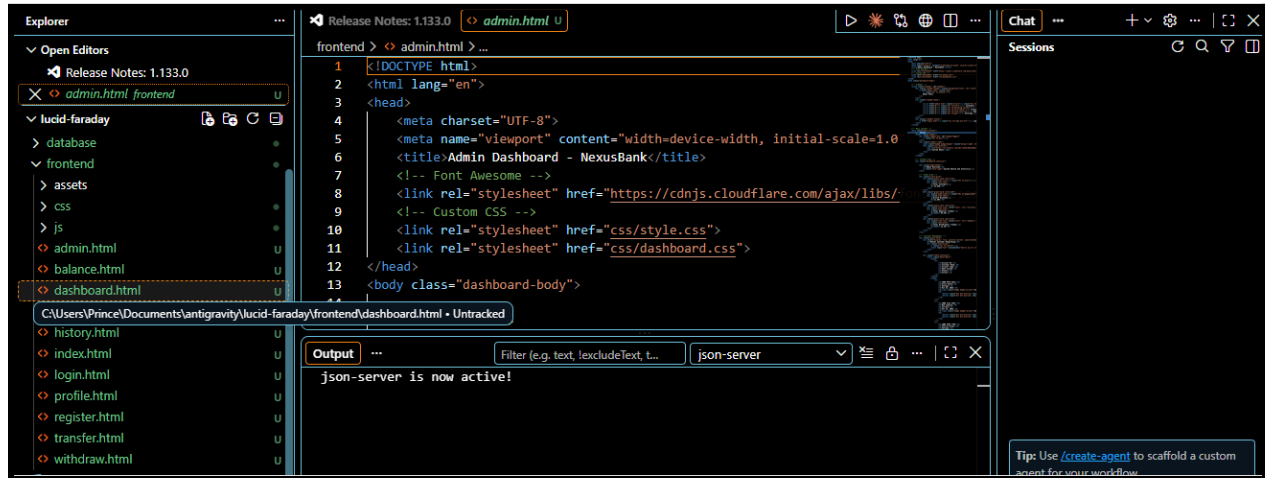
Although the current system successfully performs basic banking operations, it can be further enhanced for real-world applications. Future improvements may include integrating a MySQL database, implementing secure user authentication, adding transaction history, generating account statements, supporting online fund transfers, and providing automatic notifications. Additional security mechanisms such as encryption, role-based access, and regular data backup can also be introduced to improve the safety and scalability of the system.

REFERENCES

- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- Schildt, H. (2014). *C: The Complete Reference* (4th ed.). McGraw-Hill Education.
- Duckett, J. (2014). *HTML and CSS: Design and Build Websites*. Wiley.
- Duckett, J. (2014). *JavaScript and JQuery: Interactive Front-End Web Development*. Wiley.
- Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- Mozilla Developer Network. MDN Web Docs: HTML, CSS, and JavaScript Documentation. Mozilla.
- W3Schools. HTML, CSS, JavaScript, and C Programming Tutorials. W3Schools.
- Apache Friends. XAMPP Documentation and Apache Friends Resources. Apache Friends.
- Microsoft. Visual Studio Code Documentation. Microsoft.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson Education.
- Sommerville, I. (2015). *Software Engineering: Software Process, Requirements, Design, Testing and Maintenance*. Pearson Education.
- ISO/IEC. (2017). *ISO/IEC 25010: Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—System and Software Quality Models*. International Organization for Standardization.
- OWASP Foundation. OWASP Application Security Verification Standard (ASVS). Open Worldwide Application Security Project.

APPENDIX:

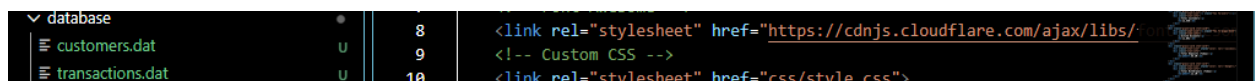
1. Frontend Files (HTML, CSS and Javascript):



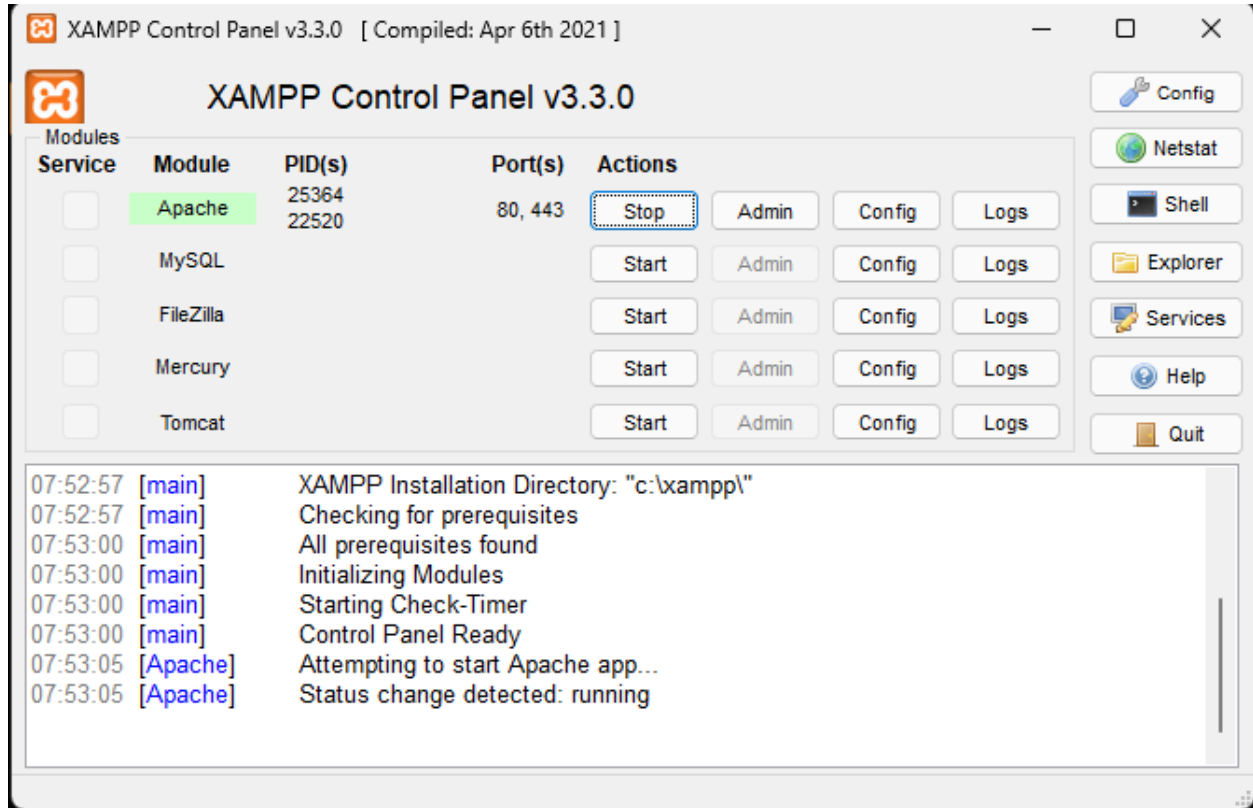
2. Backend Files (C Program and Header files)



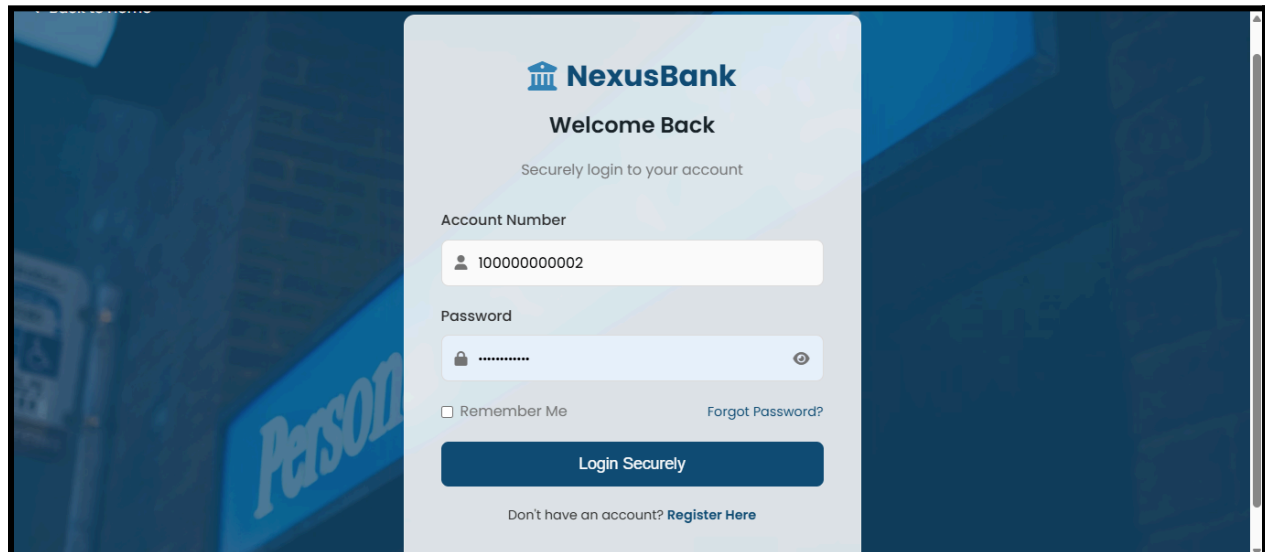
3. Database Files (.dat)



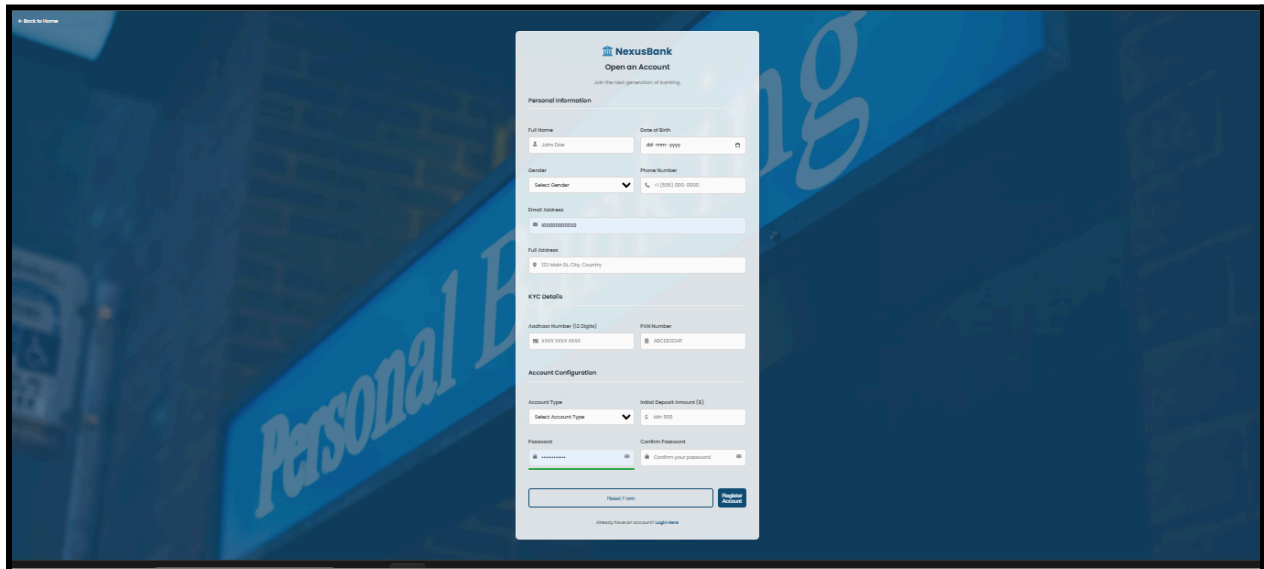
4. XAMPP Control Panel



5. Login Page:



6. Register Page for new Users



The image shows a registration form for NexusBank, titled "Open an Account". The form is divided into three main sections: Personal Information, KYC Details, and Account Configuration. The background features a blurred image of a building with a sign that says "Personal Banking".

Personal Information

Full Name: Date of Birth:

Gender: Phone Number:

Email Address:

Full Address:

KYC Details

Residence Number (12 Digits): Photo Number:

Account Configuration

Account Type: Initial Deposit Amount (\$):

Confirm Password: